Transferring Learned Repairs Between Fine-Tunes

Sergei Solovev

Independent researcher

13 October 2026

I compare two ways to reuse a repair learned for an earlier fine-tune: fit scalar coefficients over its fixed directions, or use it to initialize a repair whose directions can change. On two fine-tunes of a 67M-parameter language model, trained on different samples from the same data distribution, I repair the Dutch damage caused by training the model to predict German less accurately. At 2,048 fit tokens and a 600-step cap, donor-initialized repairs recover 11.3 percentage points (pp) more damage than the tested scalar-only search, 95% CI [+10.0, +12.7], on the primary target and 20.4 pp [+19.0, +21.7] on the replication target. Both exceed the prespecified 3 pp margin. Donor initialization is ahead of fresh fitting by 1.1 and 1.4 pp at the same budget, inside the prespecified ±2 pp equivalence band and comparable to the difference between two fresh seeds. Its advantage is larger with less data and fewer steps. On the tested grid, it reaches the quality of the first fresh seed with fewer fit-token presentations, while both families reach the reference with the same minimum number of unique tokens. For the tested donor, model, fine-tuning recipe, initializations and fitting windows, the learned repair helps most as an initialization when fitting data and steps are limited.

1 Introduction

Repeating a fine-tuning recipe on new samples can reproduce its collateral damage, so a repair learned for an earlier fine-tune may help repair the next one. The earlier fine-tune is the *donor* and its repair is the *donor repair*. We can keep the donor repair's directions fixed and fit new scalar coefficients, or use it to initialize a repair whose directions can change.

The fine-tuning procedure deliberately worsens German next-token prediction and, as a side effect, raises Dutch cross-entropy (CE), a measure of prediction loss. This provides a controlled test: the repair must restore Dutch prediction quality while preserving the intended German change. I train the donor and two target fine-tunes independently from the same base model, using the same recipe on different samples from the same Europarl data distribution. The donor repair has one rank-one update per weight matrix. The primary comparison asks how much Dutch damage is recovered by rescaling its fixed directions and by allowing those directions to change. The descriptive secondary comparisons ask how donor initialization performs against fresh and random starts at each fit budget and step cap, and what it saves (Table 1).

I fit rank-one repairs with a KL objective on the model used in Goodfire's adVersarial Parameter Decomposition (VPD) study (Bushnaq et al. 2026). The repairs operate directly on its weight matrices, independently of the decomposition.

Several methods reuse a weight update with fixed or learned scales. Task arithmetic adds or subtracts a weight difference with a single scale (Ilharco et al. 2023). RESTA adds one separately obtained safety vector to fine-tunes in several domains, using a fixed global scale in its main experiments (Bhardwaj et al. 2024). SafeGene keeps a distilled safety adapter frozen and learns one scalar per selected layer, improving on a coefficient of 1 (Wang et al. 2026, sec. 3.5 and 4.4). Such reuse can also fail. EnchTable's preprint Table II reports a medical fine-tune that RESTA leaves less safe than before (Wu et al. 2026). Other work uses a learned object to initialize new training. In SPoT, soft prompts learned on source tasks initialize the prompt of a target task (Vu et al. 2022). These studies do not compare the two uses of one transferred update on the same target at matched rank and cost. This study makes that comparison at matched rank and reports the costs of each family.

2 Setup

Model and fine-tunes. The base model is Goodfire's four-layer decoder-only model of 67M parameters, trained on a subset of the Pile (Bushnaq et al. 2026, sec. 3.1; Gao et al. 2020). Each fine-tune trains an additive update D to the 24 weight matrices of the attention and MLP blocks. Embeddings and norms stay frozen. A hinge term raises German CE, and a KL term keeps English, French, Spanish and Italian, the *retain* languages, close to the base model. The text is Europarl session text (Koehn 2005). The donor fine-tune and the two targets follow the same recipe with different seeds and training dates (method details). Dutch is held out of fine-tuning. On the readout dates the primary target's Dutch damage is 0.187 nat/token [0.178, 0.196] and the replication target's 0.189 [0.180, 0.198].

Repair. A repair adds $E_m = b_m a_m^\top$ to each of the 24 matrices of the fine-tuned model, so it has rank one per matrix. The vector a_m is the input factor and b_m is the output factor. The repair minimizes KL divergence from two teacher models across three groups of text. The fine-tuned model is the teacher on German, to preserve the intended effect, and on the retain languages. The base model is the teacher on Dutch, to undo the damage. Each term uses $D_{\mathrm{KL}}(p_{\text{teacher}} \,\|\, p_{\text{repair}})$, averaged over the group's predicted positions. The loss sums the mean KL of the three groups with equal weights. The repair norm is capped at the fine-tune norm, $\|E\|_F \le \|D\|_F$. Every target-repair family trains full-batch for 600 steps at four learning rates and keeps checkpoints as candidates for selection. Families initialized from the donor repair scale it down if needed to meet the norm cap (method details).

I first learned the donor repair from a fresh rank-one initialization (zero output vectors, random input vectors) on 73,728 prediction positions, using three learning rates and 200 minibatch steps per run. The same selected donor repair is reused for both targets.

Recovery. Recovery is the share of the fine-tune's Dutch damage that a repair X removes,

$$r_{\mathrm{nl}}(X) = 1 - \frac{\mathrm{CE}_{\mathrm{nl}}(X) - \mathrm{CE}_{\mathrm{nl}}(\text{base})}{\mathrm{CE}_{\mathrm{nl}}(\text{fine-tuned}) - \mathrm{CE}_{\mathrm{nl}}(\text{base})}.$$

Differences in recovery are reported in percentage points (pp).

In the scalar-only family, each donor repair update stays fixed and the repair is $E_m = c_m b_m a_m^\top$. Fitting the coefficients c_m trains 24 parameters. The other families train one or both factors, as listed in Table 1.

Table 1: Repair families. All have rank one per matrix and share the objective, norm cap, fit tokens, steps and selection rule. The number of trained parameters varies by family despite their equal rank. The fixed donor repair is included as a reference: it is capped and held fixed.

Family	Start	Trained (parameters)	What the comparison tests
Scalar coefficients only	capped donor repair	scalar coefficients c_m (24)	whether the donor repair's fixed directions suffice after rescaling
Donor initialization	capped donor repair	both factors (55,296)	whether training the factors improves on scalar fitting, and whether the donor repair is a useful start
Donor initialization, output only	capped donor repair	output vectors b_m (27,648)	how much of the gap closes while keeping the donor repair's input vectors fixed
Random initialization	random directions, same norm as the donor start	both factors (55,296)	whether the donor repair's directions help beyond the norm of its start
Fresh	$b_m = 0$, $a_m \sim \mathcal{N}(0, 1/d_{\mathrm{in}})$	both factors (55,296)	whether the donor repair is a better start than fitting a new repair

Varying the fit budget and the step cap tests whether donor initialization helps chiefly when fitting data and training steps are limited. Fresh fitting uses LoRA's zero-output initialization (Hu et al. 2022). Unlike the random frozen input vectors studied by Zhu et al. (2024), output-only fitting here freezes the donor repair's trained input vectors. A second seed of fresh and of random initialization, at the full budget only, shows

how far a result moves between initializations. The first fresh seed starts from the same input vectors used to initialize the donor repair, while the second uses an independent draw.

Budgets and step caps. Each target has nested fit budgets of 256, 512 and 2,048 unique predicted tokens with a 2:1:1 split of Dutch, German and retain text. These budgets count prediction positions, excluding the initial context token of each window. A separate selection and diagnostic pool contains 57,344 prediction positions, shared by all families. The targets share repair-fitting, selection and readout windows; their fine-tuning seeds and training windows differ. Checkpoints at steps $\leq$50, $\leq$100 and $\leq$200 of the same runs form step-capped cells, alongside the full 600-step runs. Each family therefore has a grid of 3 budgets $\times$ 4 step caps. The cost appendix gives the costs of selection and of the additional scalar optimization.

Selection and readout. For every family and cell, a fixed rule chooses one candidate on separate selection dates: among the candidates that keep the fine-tune's German effect and move Dutch toward the base model, it prefers higher Dutch recovery, subject to a guard against raising the CE of the retain languages (method details). The chosen repairs are read out once on 92 session dates from the validation period, separate from the project's earlier data windows (data details). The selected checkpoint can precede the cell's step cap. All cells of a target share one bootstrap over readout dates (Efron 1979), so contrasts are paired. Intervals are 95% percentile intervals.

Additional scalar optimization. A scalar-only shortfall could reflect unfinished optimization. To check this, at the 2,048-token budget, two additional projected-gradient runs optimize the same full-batch fit loss from two starts, using the convergence criteria in the appendix. The label "+ full-batch optimization" refers to these extra runs; ordinary fitting also uses full batches. Selection includes the 48 training checkpoints and the 2 converged candidates, so an ordinary checkpoint can still supply the reported result (appendix).

Interpretation rules. The primary contrast compares donor initialization with the scalar-only search with full-batch optimization at 2,048 tokens and 600 steps. An advantage of more than 3 pp is established if its whole interval lies above 3 pp, excluded if the whole interval lies below 3 pp, and undecided otherwise. I interpret this contrast on a target only if its Dutch damage is clear (the lower end of the readout interval is at least 0.025 nat/token), both selected repairs satisfy the readout constraints in Table 6, and scalar optimization converged from both starts. All other contrasts are descriptive, without multiplicity correction. Their intervals determine whether the sign is positive, negative or undecided. Five contrasts, marked in Table 3, are also assessed for equivalence within a $\pm$2 pp band. The protocol fixes both margins without a stated derivation.

3 Results

3.1 Fixed directions fall short

Both targets meet the conditions for interpreting the primary contrast (Table 6 and Table 7). Donor initialization recovers 11.3 pp [+10.0, +12.7] more damage than the scalar-only search with full-batch optimization on the primary target (Table 2) and 20.4 pp [+19.0, +21.7] more on the replication target. Both intervals lie entirely above the 3 pp margin.

The gap is nearly as large at 256 and 512 tokens, where scalar fitting received no additional optimization to convergence (Figure 1). Scalar-only fitting recovers about 87% of the damage on the primary target and 77% on the replication target, and adding the converged candidates changes this little (Table 3). Every family that trains the factors recovers at least 95.6%. A corresponding shortfall appears on the scalar-only repair's own fit tokens, measured by recovery of Dutch KL divergence from the base model rather than CE recovery (appendix).

The output-only control tests how much of this gap can be closed while keeping the donor repair's input vectors fixed. It closes most of the gap (Table 3); training both factors recovers only slightly more. Without an output-only control with frozen random input vectors, this result does not isolate the contribution of the donor repair's learned input vectors (control details).

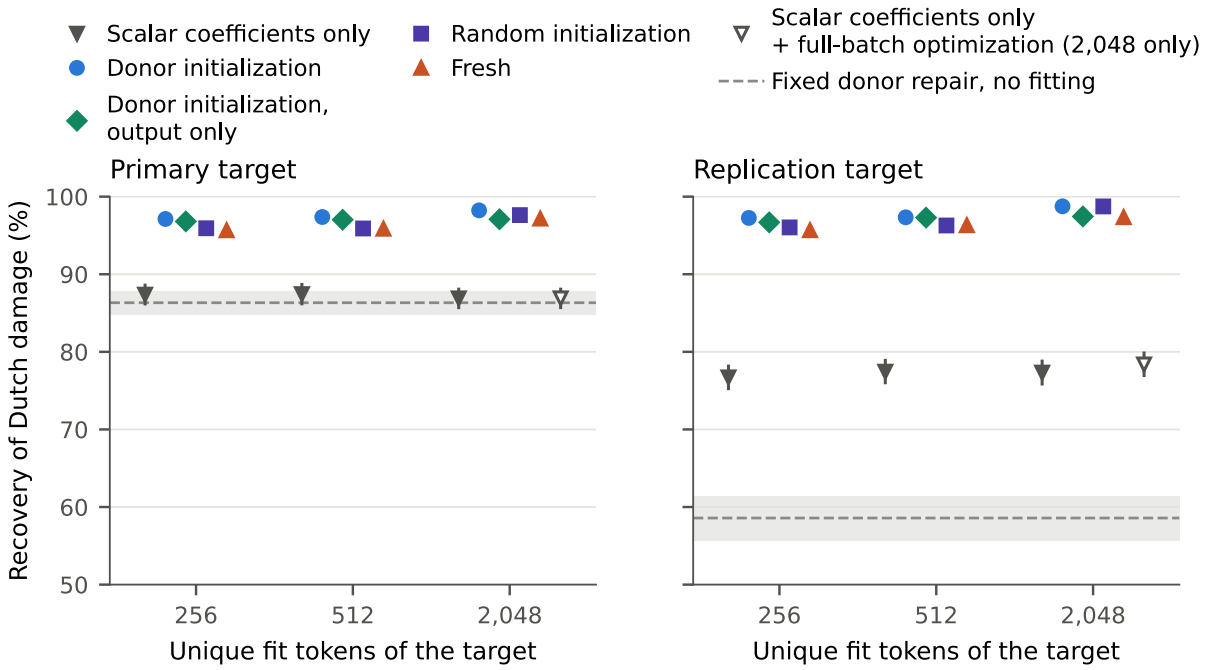


Figure 1: Recovery of the Dutch damage on the readout dates, selected from runs of up to 600 steps for each family and fit budget, with 95% bootstrap intervals over 92 dates (most are smaller than the markers). The band is the fixed donor repair and its interval. Scalar-only fitting falls well short of every family that trains the factors, at every budget and on both targets. Including the converged full-batch candidates in selection at 2,048 tokens (hollow marker) does not close the gap.

Table 2: Primary target, 2,048 fit tokens, 92 readout dates. 95% intervals from a joint bootstrap over dates (5,000 draws). Donor-initialized fitting exceeds the tested scalar-coefficient search by more than 3 percentage points. The scalar search includes 4 learning rates × 600 steps plus two full-batch optimizations, with 50 candidates in total. Selection considers both ordinary training checkpoints and converged candidates.

Repair	Recovery of Dutch damage (%)	95% CI
Donor initialization (both factors trained)	98.2	[97.6, 98.9]
Scalar coefficients only + full-batch optimization	86.9	[85.5, 88.3]
Difference (percentage points)	+11.3	[+10.0, +12.7]

3.2 The donor as an initialization

Training the directions improves on scalar rescaling. To measure the donor repair's contribution to that recovery, I compare donor initialization with fresh fitting and with random directions of the same norm.

At the full budget, donor initialization is ahead of fresh fitting by +1.1 pp [+0.8, +1.3] on the primary target and +1.4 pp [+1.1, +1.7] on the replication target (Table 3). Both intervals lie above zero and inside the ±2 pp band, so the advantage is positive but small enough to count as practical equivalence. It is comparable to the difference between two fresh seeds (+1.0 and +0.7 pp). The second fresh seed narrows both advantages further (Table 3).

Against random initialization, donor initialization is ahead at the full budget on the primary target. The sign is undecided on the replication target (Table 3).

Reducing the fit budget and step cap shows where the donor start helps more (Figure 2). With 256 fit tokens and at most 50 steps, donor initialization beats random initialization by +4.9 and +4.5 pp, and fresh fitting by

+4.4 and +4.1 pp. These advantages are larger than at the full budget. Only one fresh seed and one random seed were tested at this small budget.

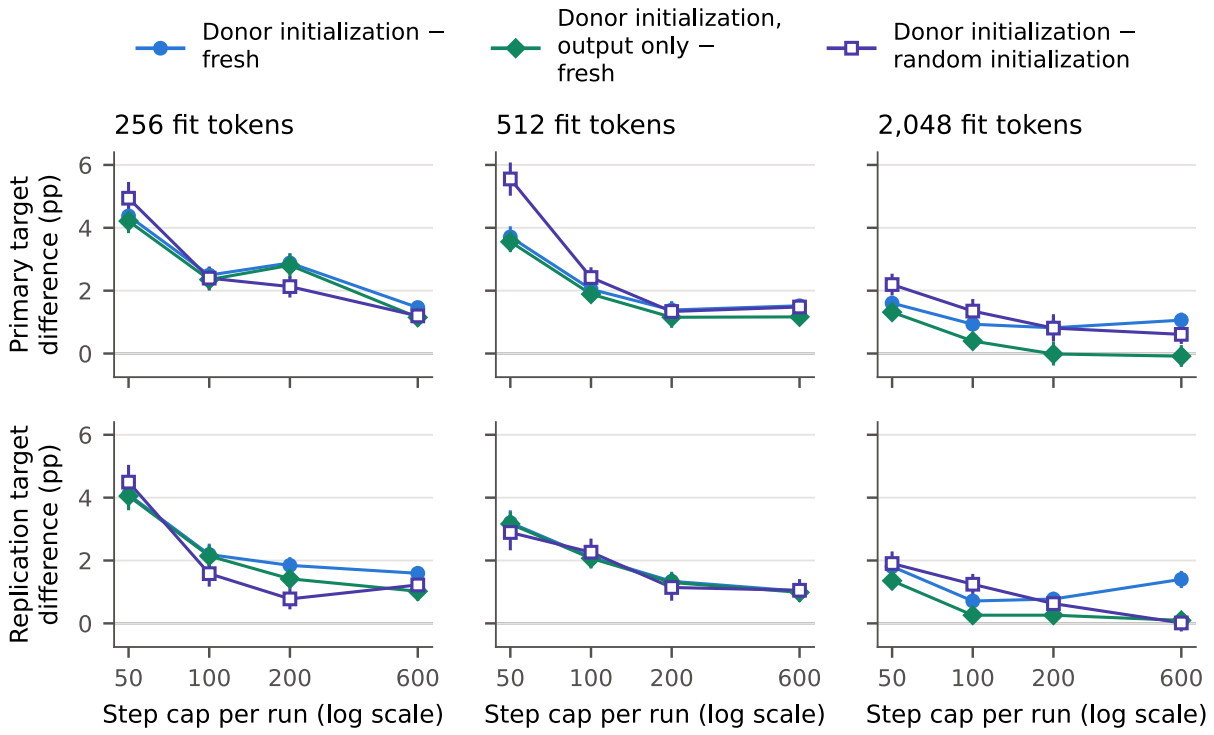


Figure 2: Paired differences of recovery at matched fit budgets and step caps: both families of a contrast have the same fit budget (columns) and step cap (horizontal axis, logarithmic, points at their true step caps), on the primary (top) and replication (bottom) target, with 95% bootstrap intervals over 92 dates for each fixed target. Donor initialization helps most in short runs on little data. With a 600-step cap and 2,048 tokens its advantage over fresh is about 1 pp and over random initialization between 0 and 1 pp.

Table 3: Secondary contrasts at the full budget (2,048 unique fit tokens, up to 600 steps): paired difference in recovery (pp) with its 95% interval, 92 readout dates per target. Rows marked '+ full-batch optimization' also include the two converged candidates. These intervals are descriptive, without multiplicity correction. The last column applies only to the five contrasts named in the protocol for an equivalence reading (primary / replication): 'yes' means that the whole interval lies inside the ±2 pp band.

Contrast (2,048 fit tokens)	Primary target	Replication target	Inside ±2 pp
Donor initialization, output only − scalar coefficients only + full-batch optimization	+10.2 [+8.9, +11.5]	+19.1 [+17.6, +20.5]	—
Donor initialization − donor initialization, output only	+1.1 [+0.7, +1.5]	+1.3 [+1.0, +1.6]	yes / yes
Donor initialization − fresh	+1.1 [+0.8, +1.3]	+1.4 [+1.1, +1.7]	yes / yes
Donor initialization, output only − fresh	−0.1 [−0.4, +0.3]	+0.1 [−0.2, +0.4]	—
Donor initialization − random initialization	+0.6 [+0.3, +0.9]	+0.01 [−0.26, +0.28]	—
Random initialization − fresh	+0.5 [+0.2, +0.7]	+1.4 [+1.0, +1.7]	yes / yes
Scalar coefficients only + full-batch optimization − fresh	−10.3 [−11.7, −8.9]	−19.0 [−20.4, −17.5]	—
Scalar coefficients only + full-batch optimization − fixed donor repair	+0.6 [+0.1, +1.1]	+19.8 [+17.8, +22.0]	—

Contrast (2,048 fit tokens)	Primary target	Replication target	Inside ±2 pp
Scalar coefficients only + full-batch optimization − scalar coefficients only	same repair selected	+1.1 [+0.8, +1.3]	—
Fresh, second seed − fresh	+1.0 [+0.7, +1.3]	+0.7 [+0.4, +0.9]	yes / yes
Random initialization, second seed − random initialization	−0.38 [−0.76, −0.02]	−1.2 [−1.5, −0.9]	yes / yes
Donor initialization − fresh, second seed	+0.04 [−0.19, +0.26]	+0.7 [+0.5, +1.0]	—

3.3 What donor initialization saves

The advantage in short runs suggests a possible saving in fitting cost. To compare costs at similar recovery, I use the first fresh seed at 2,048 tokens with a 600-step cap as a reference. A grid cell *reaches* it if the selected repair satisfies the constraints and the lower end of its paired interval against the reference lies above −2 pp. Fit-token presentations count the optimizer steps of all four learning-rate runs multiplied by the unique fit tokens.

Checkpoints are selected before readout; the cheapest qualifying budget and step-cap cell is identified afterward from the readout results. Its reported cost includes all learning-rate runs within that cell and the stated evaluation charges, but excludes exploring the other cells.

On the tested grid, donor initialization reaches this reference with one eighth of the fit-token presentations. Counted in presentations, the one-time donor cost is amortized after 11 targets. This is a descriptive projection from two tested targets; the intervals cover individual cells, not the selected cost ratio.

There is no unique-token saving: fresh fitting also reaches the reference, narrowly, with 256 fit tokens under a 600-step cap. Estimated GPU-time break-even is much later (about 151 and 106 targets), but these allocations of shared work from different runs do not establish a time saving.

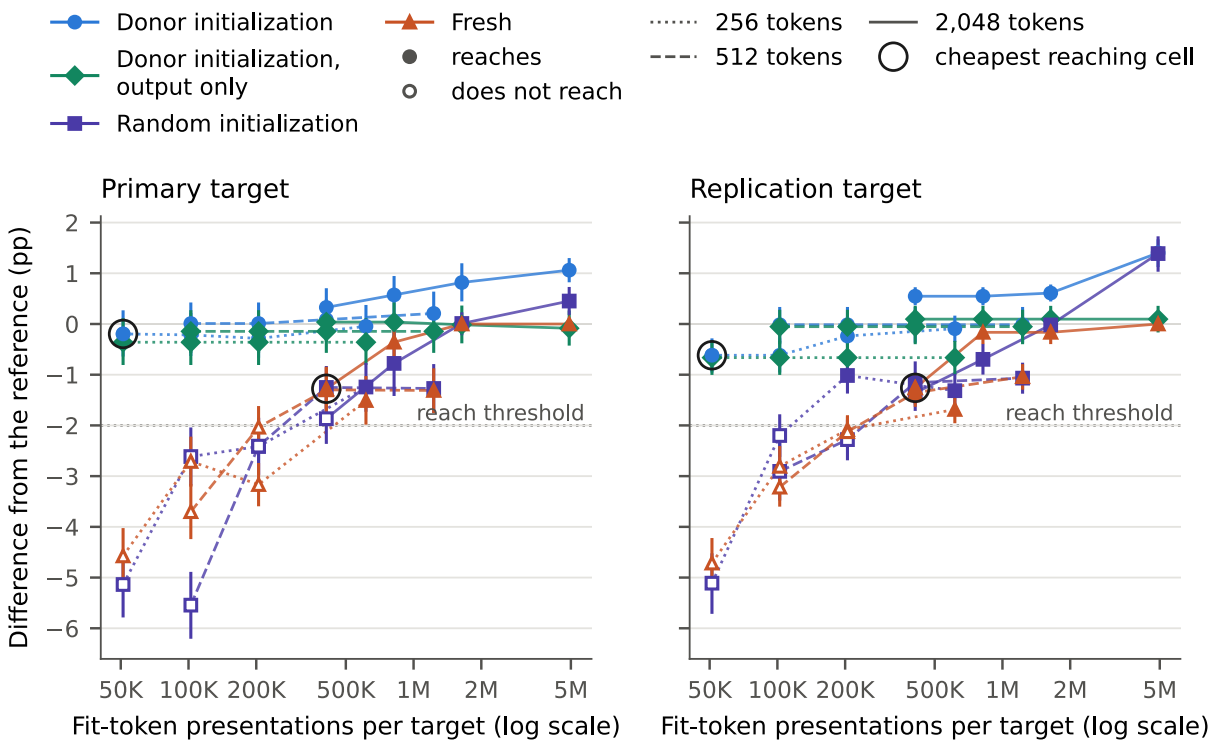


Figure 3: Quality against cost on the primary (left) and replication (right) target. Each point is a grid cell at its fit-token presentations, with its paired difference from the reference, fresh at 2,048 tokens with a 600-step cap, and its 95% bootstrap interval over 92 dates for each fixed target. Lines join the step caps 50, 100, 200 and 600. Filled markers reach the reference: chosen inside the constraints, interval above −2 pp (dotted line). Rings mark the cheapest reaching cells used for the break-even estimate: donor initialization at 256 tokens and fresh at 2,048 tokens, both with a 50-step cap (51,200 and 409,600 presentations). Scalar-only cells never reach and are not shown.

Table 4: Target-side cost of the cheapest cell of fresh fitting and of donor initialization that reaches the reference (fresh at 2,048 tokens and up to 600 steps), found separately in each currency. A cell is written as unique fit tokens / step cap. K* = donor cost / (fresh cost − donor-initialization cost) is a modelled amortization: reuse pays off for more than K* targets. The donor's one-time cost and what each currency counts and leaves out are in the cost-accounting appendix. GPU seconds are an estimate of allocated shared work, and the donor's come from a different run.

Currency	Target	Fresh	Fresh cell	Donor init.	Donor init. cell	Break-even K*
Unique tokens	Primary	57,600	256 / 600	57,600	256 / 600	no per-target saving
	Replication	57,600	256 / 600	57,600	256 / 600	no per-target saving
Fit-token presentations	Primary	409,600	2,048 / ≤50	51,200	256 / ≤50	10.3
	Replication	409,600	2,048 / ≤50	51,200	256 / ≤50	10.3
GPU seconds (estimate)	Primary	16.8	2,048 / ≤50	14.5	256 / ≤50	151.4
	Replication	17.1	2,048 / ≤50	13.9	256 / ≤50	105.9

4 Limits of these conclusions

The experiment uses one donor, one recipe and one model, with each target analysed separately. The bootstrap quantifies variation across readout dates for those targets. Variation across fine-tunes, donors and recipes

remains unmeasured. Second seeds were run at the full budget only, leaving short-run advantages unassessed against seed variation.

The scalar-only shortfall applies to the tested search and norm cap (optimization details). The mechanism and the contribution of the donor repair's input vectors remain unresolved (control details). The cost projection uses the first fresh seed as its reference and depends on the chosen ±2 pp quality criterion and tested grid (cost accounting).

The repair addresses collateral Dutch CE damage after a fine-tune that deliberately worsens German prediction. Safety alignment and transfer of a new skill remain outside the study's scope.

5 Conclusion

For this model, donor and recipe, training the donor repair's directions outperformed the tested scalar-only search, including its converged full-batch candidates. Donor initialization helped most with little fitting data and few steps. At the full budget, its advantage over fresh fitting was positive but within the ±2 pp equivalence band and comparable to fresh-seed differences. On the tested grid, its cheapest reaching cell used one eighth as many fit-token presentations as fresh fitting's cheapest reaching cell, while both families reached the reference with 256 unique fit tokens.

AI assistance. I used OpenAI Codex and Anthropic Claude for research planning, code development, data analysis, and manuscript preparation. Scientific Agent Skills (Kassis et al. 2026) supplied the scientific-writing, scientific-critical-thinking, peer-review and citation-management workflows used to review organization, claims and references.

References

Barzilai, Jonathan, and Jonathan M. Borwein. 1988. "Two-Point Step Size Gradient Methods." *IMA Journal of Numerical Analysis* 8 (1): 141–48. https://doi.org/10.1093/imanum/8.1.141.

Bhardwaj, Rishabh, Duc Anh Do, and Soujanya Poria. 2024. "Language Models Are Homer Simpson! Safety Re-Alignment of Fine-Tuned Language Models Through Task Arithmetic." *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Bangkok, Thailand), 14138–49. https://doi.org/10.18653/v1/2024.acl-long.762.

Birgin, Ernesto G., José Mario Martínez, and Marcos Raydan. 2000. "Nonmonotone Spectral Projected Gradient Methods on Convex Sets." *SIAM Journal on Optimization* 10 (4): 1196–211. https://doi.org/10.1137/S1052623497330963.

Bushnaq, Lucius, Dan Braun, Oliver Clive-Griffin, et al. 2026. *Interpreting Language Model Parameters*. Goodfire and MATS. https://www.goodfire.com/research/interpreting-lm-parameters.

Efron, Bradley. 1979. "Bootstrap Methods: Another Look at the Jackknife." *The Annals of Statistics* 7 (1): 1–26. https://doi.org/10.1214/aos/1176344552.

Gao, Leo, Stella Biderman, Sid Black, et al. 2020. *The Pile: An 800GB Dataset of Diverse Text for Language Modeling*. https://doi.org/10.48550/arXiv.2101.00027.

Hu, Edward J., Yelong Shen, Phillip Wallis, et al. 2022. "LoRA: Low-Rank Adaptation of Large Language Models." *International Conference on Learning Representations*. https://openreview.net/forum?id=nZeVKeeFYf9.

Ilharco, Gabriel, Marco Tulio Ribeiro, Mitchell Wortsman, et al. 2023. "Editing Models with Task Arithmetic." *International Conference on Learning Representations*. https://openreview.net/forum?id=6t0Kwf8-jrj.

Kassis, Timothy, Vinayak Agarwal, Yuhuan He, Darshil Patel, and Aubrey M. Brueckner. 2026. *Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents*. https://doi.org/10.48550/arXiv.2609.00065.

Kingma, Diederik P., and Jimmy Ba. 2015. "Adam: A Method for Stochastic Optimization." *International Conference on Learning Representations*. https://arxiv.org/abs/1412.6980.

Koehn, Philipp. 2005. "Europarl: A Parallel Corpus for Statistical Machine Translation." *Proceedings of Machine Translation Summit X: Papers* (Phuket, Thailand), 79–86. https://aclanthology.org/2005.mtsummit-papers.11/.

Vu, Tu, Brian Lester, Noah Constant, Rami Al-Rfou', and Daniel Cer. 2022. "SPoT: Better Frozen Model Adaptation Through Soft Prompt Transfer." *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Dublin, Ireland), 5039–59. https://doi.org/10.18653/v1/2022.acl-long.346.

Wang, Yanghan, Zhiqiang Kou, Fu Feng, Jing Wang, and Xin Geng. 2026. *SafeGene: Reusable Adapters for Transferable Safety Alignment*. https://doi.org/10.48550/arXiv.2606.06519.

Wu, Jialin, Kecen Li, Zhicong Huang, Xinfeng Li, Xiaofeng Wang, and Cheng Hong. 2026. "EnchTable: Unified Safety Alignment Transfer in Fine-Tuned Large Language Models." *2026 IEEE Symposium on Security and Privacy (SP)*, 3358–76. https://doi.org/10.1109/SP63933.2026.00072.

Zhu, Jiacheng, Kristjan Greenewald, Kimia Nadjahi, et al. 2024. "Asymmetry in Low-Rank Adapters of Foundation Models." *Proceedings of the 41st International Conference on Machine Learning*, Proceedings of Machine Learning Research, vol. 235: 62369–85. https://proceedings.mlr.press/v235/zhu24c.html.

A Method details

Fine-tunes. Each fine-tune learns full-matrix additive updates D_m to the query, key, value, output and two MLP matrices of each layer (24 matrices). Adam (Kingma and Ba 2015) runs at learning rate 3e-6 for 200 steps on the per-window hinge $\max(0, 1 - \Delta\mathrm{CE}_{\mathrm{de}})$ (1-nat margin) plus retain-language KL with weight 1. The checkpoint is chosen on German validation dates.

Repairs. Ordinary donor and target fitting uses Adam. Donor checkpoints are saved every 25 steps and selected on the donor's validation windows by the rule below. Target repairs save checkpoints every 50 steps at the four learning rates in Table 5, giving 48 candidates per family. Donor starts are capped as $\min(1, \|D\|_F / \|E_{\mathrm{donor}}\|_F)\, E_{\mathrm{donor}}$; fresh initialization is defined in Table 1.

Table 5: Learning-rate grids of the repair families.

Family	Learning rates
Scalar coefficients only	3e-3, 1e-2, 3e-2, 1e-1
Donor initialization, donor initialization output only, random initialization and fresh	1e-4, 3e-4, 1e-3, 3e-3

The same grid serves the 50-step cap. Several output-only cells and the donor-initialized replication cell at 512 tokens select the first checkpoint at the lowest rate; reuse alone can explain repeated values across caps. Advantages over fresh and random starts vary nonmonotonically with budget and cap. No output-only control with frozen random input vectors was run.

Selection. Feasibility requires a German-damage ratio (with repair / without repair) of 0.9–1.1, German KL drift from the fine-tuned model at most 0.05 times the base model's KL from that fine-tune, and Dutch movement toward the base model. Feasible candidates are ranked by Dutch recovery, then smaller norm, then listing order. Among the top five, the first raising no retain language's CE by more than 0.01 nat/token is chosen.

Data. Portuguese, the seventh language, is diagnostic only on selection and readout dates. Its selection windows count toward token and evaluation costs, but it enters neither the loss nor selection constraints.

A readout date was ineligible if found in recorded project data, pilot or run files, or if its window duplicated text already in use. All dates passed; prior model exposure remains unresolved. Selection and readout use 32 and 92 dates, respectively, each providing one 257-token window (256 predictions) from the same turn in every language. Target fit windows and selection dates are separate from the donor's. The fit budgets are nested: at 256 tokens, one Dutch window contributes 128 predictions, one German 64, and four retain windows 16 each; 512 doubles these prediction counts. At 2,048 tokens, four Dutch and two German windows contribute 256 predictions each, and one window per retain language contributes 128.

Study record. According to my records, a dated protocol fixed rules, margins and families before readout-data construction and target fine-tuning. An author-dated amendment before any target fine-tune or readout fixed the readout data and clarified the primary interpretation without changing decision rules. The unpublished protocol, amendment and project log lack independent timestamps, so this chronology is not verifiable preregistration.

Uncertainty. Within each target, 5,000 joint date resamples cover all budgets and step caps; each computes recovery ratios from mean damages. Readout constraints use point estimates on all dates. Equivalence is

assessed only for the five contrasts marked in Table 3; it remains unestablished between donor and random initialization at every budget and cap.

B Checks behind the primary reading

All 2,048-token cells and the fixed donor repair satisfy the readout constraints (Table 6). On the fit tokens, the selected scalar-only repair removes 87.0% and 79.2% of the fine-tune's Dutch KL from the base model on the primary and replication targets; every family that trains the factors removes at least 98.4%.

Table 6: Readout of the 2,048-token cells (up to 600 steps) and the fixed donor repair on the 92 readout dates. Rows marked '+ full-batch optimization' also include the two converged candidates. KL recovery (%) is the share of the fine-tune's Dutch KL from the base model that the repair removes. German effect kept is the ratio of German CE damage with and without the repair (required 0.9–1.1). German drift is the repair's German KL from the fine-tuned model relative to the base model's KL from the fine-tuned model (required at most 0.05). Worst retain change is the largest CE change in English, French, Spanish or Italian against the fine-tuned model in nat/token (required at most +0.01).

Target	Repair	Recovery (%) [95% CI]	KL recovery	German effect kept	German drift	Worst retain change
Primary	Scalar coefficients only	86.9 [85.5, 88.3]	86.7	1.005	0.0003	+0.0003
Primary	Scalar coefficients only + full-batch optimization	86.9 [85.5, 88.3]	86.7	1.005	0.0003	+0.0003
Primary	Donor initialization	98.2 [97.6, 98.9]	98.3	0.996	0.0005	−0.0007
Primary	Donor initialization, output only	97.1 [96.4, 97.7]	98.1	1.003	0.0003	−0.0006
Primary	Random initialization	97.6 [97.0, 98.3]	98.2	0.988	0.0005	−0.0009
Primary	Random initialization, second seed	97.2 [96.5, 98.0]	97.5	0.990	0.0005	−0.0012
Primary	Fresh	97.2 [96.5, 97.8]	97.7	0.990	0.0005	−0.0014
Primary	Fresh, second seed	98.2 [97.5, 98.9]	97.8	0.985	0.0006	−0.0015
Primary	Fixed donor repair	86.3 [84.8, 87.9]	84.4	0.999	0.0002	−0.0001
Replication	Scalar coefficients only	77.3 [75.7, 79.0]	76.5	1.007	0.0002	+0.0003
Replication	Scalar coefficients only + full-batch optimization	78.4 [76.7, 80.0]	77.8	1.007	0.0003	+0.0004
Replication	Donor initialization	98.8 [98.1, 99.4]	98.6	1.001	0.0003	−0.0005
Replication	Donor initialization, output only	97.4 [96.8, 98.1]	98.2	1.007	0.0002	−0.0006
Replication	Random initialization	98.7 [98.0, 99.4]	97.9	0.990	0.0004	−0.0010
Replication	Random initialization, second seed	97.5 [96.8, 98.3]	97.8	0.996	0.0004	−0.0009
Replication	Fresh	97.4 [96.7, 98.0]	98.1	0.994	0.0004	−0.0012
Replication	Fresh, second seed	98.0 [97.4, 98.6]	98.4	0.990	0.0004	−0.0012
Replication	Fixed donor repair	58.6 [55.6, 61.4]	48.0	1.009	0.0003	−0.0001

The additional scalar search uses $e_m = c_m \|b_m\| \|a_m\|$, so $\|E\|_F = \|e\|_2$ and radial scaling projects exactly onto the norm cap. It combines the spectral step of Barzilai and Borwein (1988), the projected-gradient scheme of Birgin et al. (2000) and a monotone Armijo line search for at most 300 iterations, starting from the initial coefficients and the lowest-fit-loss checkpoint.

Convergence requires all losses and gradients, including line-search trials, to be finite, initial training/export loss agreement within 1e-5 nat/token, and **at least one** stopping condition: projected-gradient norm at most 1e-4 of its reference; relative fit-loss decrease below 1e-7 for five iterations; or the line-search floor with stationarity at most 1e-3 of that reference. Both starts use the projected-gradient reference at the initial coefficients. Agreement between final losses is descriptive, not a convergence criterion.

Table 7: The scalar runs met the declared stopping criteria from both starts on both targets. The end losses differ by a relative 2.0e-5 on the primary and 1.1e-6 on the replication target.

Target	Start	Stop	Iterations	Fit loss, start → end	Stationarity / reference
Primary	initial scalar coefficients	relative decrease	107	0.02516 → 0.02070	0.0062
Primary	best checkpoint	relative decrease	84	0.02129 → 0.02070	0.0019
Replication	initial scalar coefficients	line-search floor	69	0.08323 → 0.03682	0.0002
Replication	best checkpoint	relative decrease	49	0.03870 → 0.03682	0.0004

This tests four learning rates for 600 steps plus optimization from two starts under $\|E\|_F \leq \|D\|_F$. Other assignments of the 24 scalar coefficients and larger repair norms remain untested.

C Cost accounting

For each cost measure, let C^* be a family's cheapest reaching cost on the tested grid. Reuse pays off for more than

$$K^* = \frac{C_{\text{donor}}}{C^*(\text{fresh}) - C^*(\text{donor initialization})}$$

targets (Table 4). Both cheapest cells in fit-token presentations use the grid's shortest step cap, and donor initialization uses its smallest budget; the one-eighth ratio follows from 2,048/256. The donor's 3,686,400 presentations give $K^* \approx 10.3$: an 11-target projection from two tested targets. Neither scalar-only fitting nor the fixed donor repair reaches the reference (Figure 3). A random start saves no presentations on the primary target; on the replication target it gives $K^* = 18$, charged with the full donor cost despite using only the donor repair's norm.

Unique tokens count fit tokens plus 57,344 selection positions (32 dates × 7 languages × 256). Presentations count fit tokens × optimizer steps across all four learning rates up to the cap, excluding selection forward passes. GPU seconds combine training time prorated to the cap with selection-evaluation time allocated by candidate count. For the expanded scalar search, presentations and GPU seconds also include both additional runs and evaluation of the 48 checkpoints used to choose the second start. All cells exclude base/fine-tuned readouts and the damage check before fitting.

The donor repair costs 131,072 unique tokens (73,728 fit plus selection), 3,686,400 presentations (3 learning rates × 200 steps × 6,144 tokens per minibatch), and an estimated 336 GPU seconds for training, selection and readout. The donor fine-tune is excluded because the donor model is assumed to exist.

The reference is the first fresh seed (Section 3.3); its paired interval against itself is [0, 0]. At 256 tokens and a 600-step cap, fresh fitting's lower bounds are −1.985 pp (primary) and −1.957 pp (replication), narrowly above the −2 pp threshold. This cell sets its lowest unique-token cost. Break-even depends on that quality threshold, the grid and reference seed; sensitivity to another seed remains unevaluated.

D Reproducibility and availability

Code, saved measurements, and instructions for replaying analysis and running the model procedures are on GitHub.